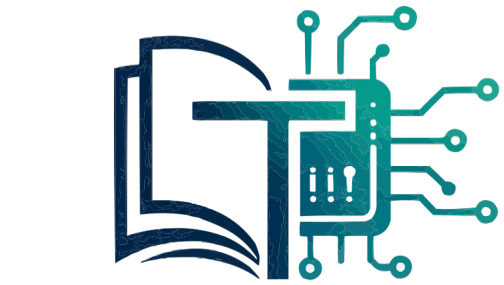


Digital Chip Design Program

RTL Design

Almas Alshubrumi
Atheer Al-harbi
Mjd AL-harbi
Maha Omar
Jood Alomair

29/7/2026

Controller & Datapath

Version 1.0

Task #14.1.1

```
module alu #(
       parameter ALU_WIDTH =4
       )(
     input logic [ALU_WIDTH-1:0] op1,
    input logic [ALU_WIDTH-1:0] op2,
    input logic [1:0] opcode,
    output logic [ALU_WIDTH-1:0] alu_result

   );

   always_comb begin
    case(opcode)
     2'b00: alu_result = op1 + op2;
     2'b01: alu_result = op1 - op2;
     2'b10: alu_result = op1 & op2;
     2'b11: alu_result = op1 | op2;
     default: alu_result = 'bx;

   endcase
   end

endmodule
```

```
module program_counter # (
          parameter PROG_VALUE = 16
     )(
       input logic clk,
      input logic rstn,
      output logic [($clog2(PROG_VALUE))-1:0]
counter



     );
  always_ff @(posedge clk or negedge rstn )
begin
      if (!rstn)
     counter <= 0;
   else
     counter <= counter +1;
 end

endmodule
```

```
module register_file #( parameter
REGF_WIDTH = 4)(
input logic clk,rstn,
input logic [1:0] rs1,
input logic [1:0] rs2,
input logic [1:0] rd,
input logic [REGF_WIDTH-1:0] alu_result,
output logic [REGF_WIDTH-1:0] op1,
output logic [REGF_WIDTH-1:0] op2
);
logic [REGF_WIDTH-1:0] x [0:3];
always @ (posedge clk ,negedge rstn)begin
if(!rstn)
for(int i = 0;i<REGF_WIDTH;i++)
x[i]<=0;
//alu result
case(rd)
2'b01: x[1]<= alu_result;
2'b10: x[2]<= alu_result;
2'b11 : x[3]<= alu_result;
default:begin
x[1]<= x[1];
x[2]<= x[2];
x[2]<= (x[2]);
end
endcase
end
//rs2
always_comb begin
case(rs2)
2'b00:op2=x[0];
2'b01:op2=x[1];
2'b10:op2=x[2];
2'b11:op2=x[3];
endcase
end
//rs1
always_comb begin
case(rs1)
2'b00:op1=x[0];
2'b01:op1=x[1];
2'b10:op1=x[2];
2'b11:op1=x[3];
endcase
end
endmodule
```

Task #14.1.1

```
module instruction_memory #(parameter
IMEM_DEPTH=16,
FILE_NAME="C:\\Users\alfra\Desktop\trinin
g\baby processor\machin_code.mem")
(  input
logic[$clog2(IMEM_DEPTH)-1:0]address,
  output logic [7:0] instruction );
  logic [7:0] imem [IMEM_DEPTH-1:0];
  initial $readmemb(FILE_NAME,imem);
  assign instruction=imem[address];
endmodule
```

```
module datapath #(
  parameter
    IMEM_DEPTH = 16,
    PROG_VALUE = 16,
    REGF_WIDTH = 4,
    ALU_WIDTH = 4,
    FILE_NAME =
"C:\Users\almas\Desktop\test\Almas\day17
\task#14\machine_code.mem"
)(
 input logic clk,
  input logic rstn,
input logic [1:0] rs1,
input logic [1:0] rs2,
input logic [1:0] rd,
input logic [1:0] opcode,

output logic [7:0] instruction
);
logic [$clog2(IMEM_DEPTH)-1:0] address;
logic [ALU_WIDTH-1:0] op1;
logic [ALU_WIDTH-1:0] op2;
logic [ALU_WIDTH-1:0] alu_result;
program_counter #(
  .PROG_VALUE(PROG_VALUE)
) pc_ins (
  .clk(clk),
  .rstn(rstn),
  .counter(address)
);
instruction_memory #(
 .IMEM_DEPTH(IMEM_DEPTH),
 .FILE_NAME(FILE_NAME)
) rom_ins (
 .address(address),
 .instruction(instruction)
);

register_file #(
  .REGF_WIDTH(REGF_WIDTH)
) rf_ins (
  .clk(clk),
  .rstn(rstn),
  .rs1(rs1),
  .rs2(rs2),
  .rd(rd),
  .alu_result(alu_result),
  .op1(op1),
  .op2(op2)
);
alu #(
  .ALU_WIDTH(ALU_WIDTH)
) alu_ins (
  .op1(op1),
  .op2(op2),
  .opcode(opcode),
  .alu_result(alu_result)
);
endmodule
```

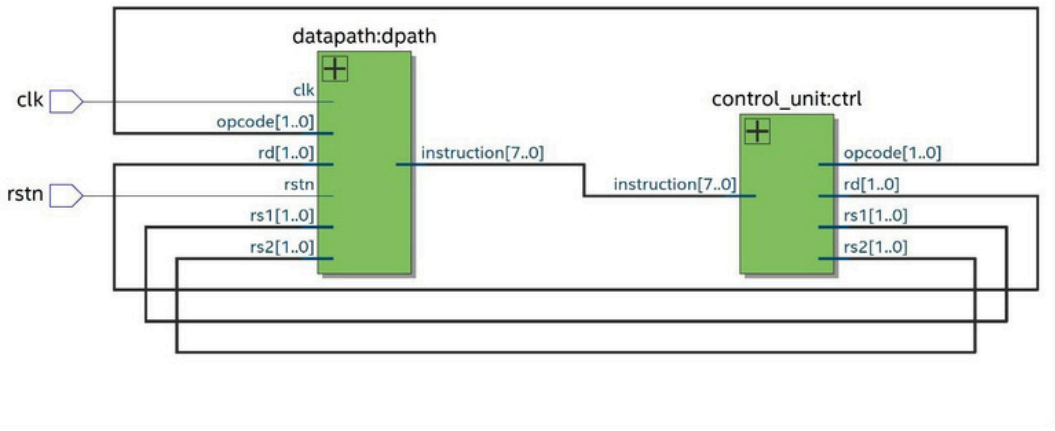
Task #14.1.2

control unit:

```
module control_unit(
input logic [7:0] instruction,
output logic [1:0] rs1,
output logic [1:0] rs2,
 output logic [1:0] rd,
output logic [1:0] opcode
);

assign rd = instruction[7:6];
assign rs2 = instruction[5:4];
assign rs1 = instruction [3:2];
assign opcode = instruction [1:0];

endmodule
```

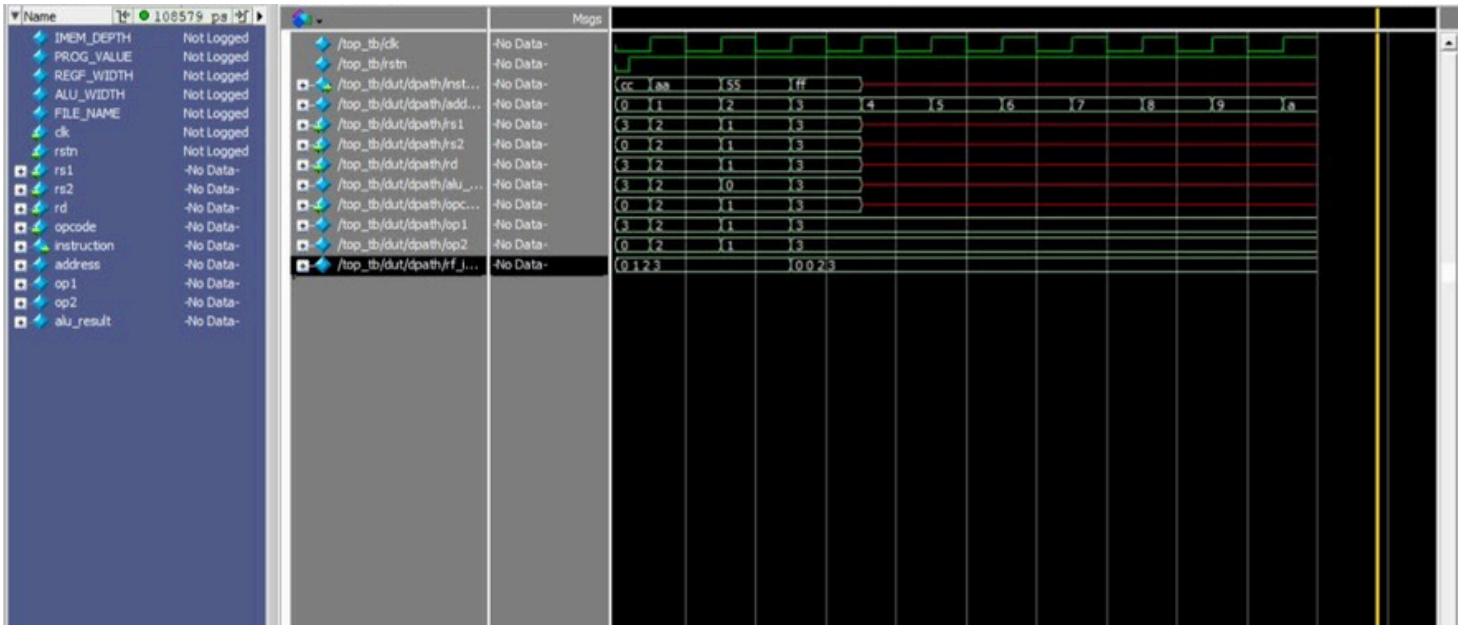
Task #14.1.3

```
module top2 #(
    parameter
        IMEM_DEPTH = 16,
        PROG_VALUE = 16,
        REGF_WIDTH = 4,
        ALU_WIDTH = 4,
        FILE_NAME =
"C:\Users\almas\Desktop\test\Almas\day17\task#14\machine_code.mem"
)(
    input logic clk,
    input logic rstn
);
logic [7:0] instruction;
logic [1:0] rs1;
logic [1:0] rs2;
logic [1:0] rd;
logic [1:0] opcode;
datapath #(
    .IMEM_DEPTH(IMEM_DEPTH),
    .PROG_VALUE(PROG_VALUE),
    .REGF_WIDTH(REGF_WIDTH),
    .ALU_WIDTH(ALU_WIDTH),
    .FILE_NAME(FILE_NAME)
) dpath (
.clk(clk),
.rstn(rstn),

.rs1(rs1),
.rs2(rs2),
.rd(rd),
.opcode(opcode),

.instruction(instruction)
);
control_unit ctrl (
.instruction(instruction),

.rs1(rs1),
.rs2(rs2),
.rd(rd),
.opcode(opcode)
);
endmodule
```



Task #14.1.4

```
`timescale 1ns / 1ps

module top2_tb;

        logic clk;
        logic rstn;

        top2 #(.FILE_NAME("C:\\Users\\JOUDA\\Desktop\\Baby B
processor\\machine_code.mem")) t(
            .clk(clk), .rstn(rstn)
        );


        initial begin
            rstn = 0;
            clk = 0;
            #2 rstn = 1;
        end

        always #5 clk = ~clk;

        initial begin
            #100 $finish;
        end
endmodule
```



Work distribution

Section	Almas	Atheer	Mjd	Maha	Jood
Report	✔	✔	✔	✔	✔
Task14.1	✔	✔	✔	✔	✔
Task14.2	✔	✔	✔	✔	✔
Task14.3	✔	✔	✔	✔	✔